

Text Preprocessing: Stop Word Removal, Stemming and Lemmatization

Text Pre-processing/Normalization: Stop Word Removal

- Remove commonly used words in a language
 - *a, the, is, are* etc.
- These are the most common words in any language (like articles, prepositions, pronouns, conjunctions, etc)
- They do not add much information to the text. Removing them improves computational efficiency.
- Removal of such words does not show any negative consequences on the model we train for our task.

“I have never tasted a pizza like that before.”

Word Tokenization

["I", "have", "never", "tasted", "a", "pizza", "like", "that", "before"]

Identify stop words

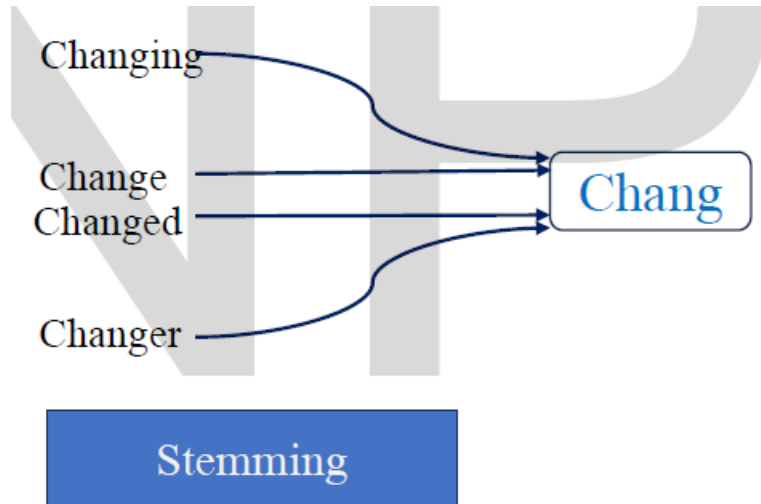
["I", "have", "a", "like", "that", "before"]

Final Output

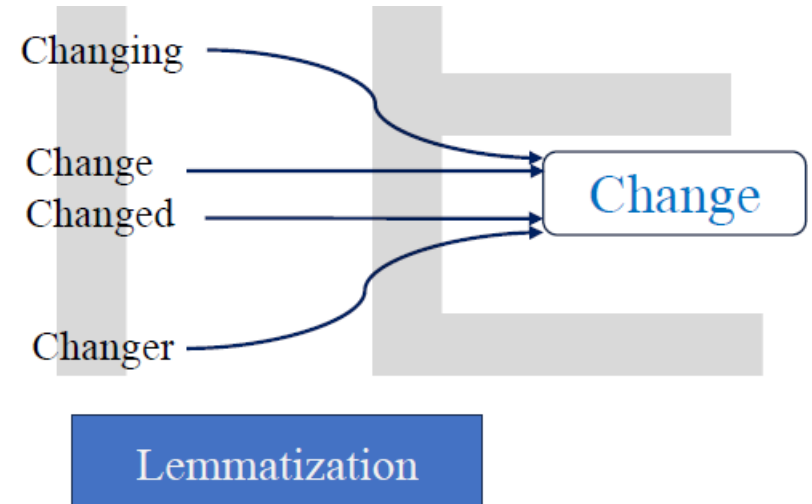
“never tasted pizza”

Text Pre-processing/Normalization: Stemming and Lemmatization

- Stemming is a technique where words are reduced to their base or root form by removing suffixes or prefixes.
- Lemmatization is a sophisticated technique that transforms words to their base forms, known as lemmas, considering the word's meaning and context.
- **Lemmatization:** determining that two words have the same root, despite their surface differences
 - sang, sung, and sings are forms of sing



Doesn't have linguistic knowledge



Have linguistic knowledge

Text Pre-processing/Normalization: Casing

- **Casing:** Lowercase all words or not
 - *With pretrained language models, besides casing, we do none of the other steps*
- After text normalization, most tokenizers are **irreversible**
- We cannot recover the raw text definitively from the tokenized output

Python Example: Stop Word Removal

- code for removing stop words using the NLTK library:
 - `text = "S&P and NASDAQ are the two most popular indices in US"`
 - **`from nltk.corpus import stopwords`**
 - **`from nltk.tokenize import word_tokenize`**
 - `stop_words = set(stopwords.words('english'))`
 - `text_tokens = word_tokenize(text)`
 - `tokens_without_sw= [word for word in text_tokens if not word in stop_words]`
 - `print(tokens_without_sw)`
 - Output:
 - ['S', '&', 'P', 'NASDAQ', 'two', 'popular', 'indices', 'US']
- The `stop_words.words('english')` is a set of default stop words for the English language model in NLTK.

Python Example: Stemming

- `text = "It's a Stemming testing"`
- `parsed_text = word_tokenize(text)`
- *# Initialize stemmer.*
- **`from nltk.stem.snowball import SnowballStemmer`**
- `stemmer = SnowballStemmer('english')`
- *# Stem each word.*
- **`[(word, stemmer.stem(word)) for i, word in enumerate(parsed_text)]`**
- **`if word.lower() != stemmer.stem(parsed_text[i])`**
 - Output
 - `[('Stemming', 'stem'), ('testing', 'test')]`

Python Example: Lemmatization

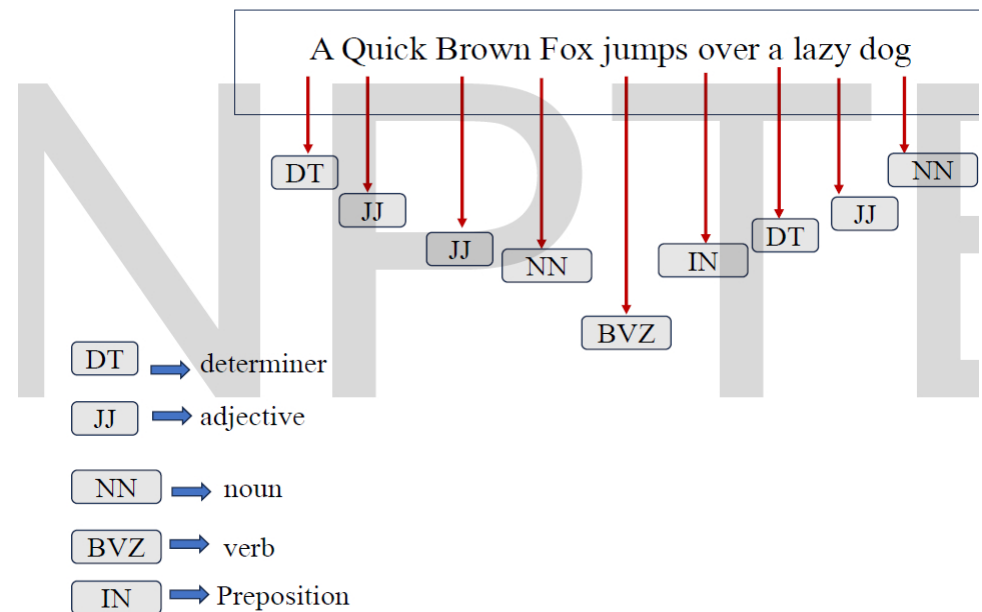
- `text = "This world has a lot of faces "`
- **`from textblob import Word`**
- `parsed_data= TextBlob(text).words`
- `[(word, word.lemmatize()) for i, word in enumerate(parsed_data)]`
- **`if word != parsed_data[i].lemmatize()]`**
 - Output
 - `[('has', 'ha'), ('faces', 'face')]`

Text Preprocessing: Parts of Speech (PoS) Tagging and Named Entity Recognition (NER)

Text Pre-processing/Normalization:

Part of Speech (PoS) Tagging

- PoS involves assigning a grammatical category (such as noun, verb, adjective, etc.) to each word in a sentence.
- Helps machines understand the syntactic structure (grammar and structure - the rules for arranging words/symbols in a sentence) of sentences, which is crucial for tasks like:
 - Machine Translation
 - Information extraction
 - Question answering
 - Text to speech conversion
- Eight basic parts of speech: noun, verb, pronoun, preposition, adverb, conjunction, participle, and article. (Ref.: Dionysius Thrax of Alexandria (c. 100 B.C.))
- Words often have more than one POS.
 - VERB: (**Book** that flight)
 - NOUN: (Hand me that **book**).
- The goal of POS-tagging is to resolve ambiguities, choosing the proper tag for the context.



PoS Tagging: Popular tag-set

- An English-specific tagset, the **Penn Treebank tagset** ([Marcus et al., 1993](#)), has been used to label many syntactically annotated corpora like the Penn Treebank corpora
- It has 36 primary POS tags

Tag	Description	Example	Tag	Description	Example	Tag	Description	Example
CC	coordinating conjunction	<i>and, but, or</i>	PDT	predeterminer	<i>all, both</i>	VBP	verb non-3sg present	<i>eat</i>
CD	cardinal number	<i>one, two</i>	POS	possessive ending	<i>'s</i>	VBZ	verb 3sg pres	<i>eats</i>
DT	determiner	<i>a, the</i>	PRP	personal pronoun	<i>I, you, he</i>	WDT	wh-determ.	<i>which, that</i>
EX	existential 'there'	<i>there</i>	PRP\$	possess. pronoun	<i>your, one's</i>	WP	wh-pronoun	<i>what, who</i>
FW	foreign word	<i>mea culpa</i>	RB	adverb	<i>quickly</i>	WP\$	wh-possess.	<i>whose</i>
IN	preposition/ subordin-conj	<i>of, in, by</i>	RBR	comparative adverb	<i>faster</i>	WRB	wh-adverb	<i>how, where</i>
JJ	adjective	<i>yellow</i>	RBS	superlatv. adverb	<i>fastest</i>	\$	dollar sign	<i>\$</i>
JJR	comparative adj	<i>bigger</i>	RP	particle	<i>up, off</i>	#	pound sign	<i>#</i>
JJS	superlative adj	<i>wildest</i>	SYM	symbol	<i>+, %, &</i>	“	left quote	<i>‘ or “</i>
LS	list item marker	<i>1, 2, One</i>	TO	“to”	<i>to</i>	”	right quote	<i>’ or ”</i>
MD	modal	<i>can, should</i>	UH	interjection	<i>ah, oops</i>	(	left paren	<i>[, (, {, <</i>
NN	sing or mass noun	<i>llama</i>	VB	verb base form	<i>eat</i>	)	right paren	<i>],), }, ></i>
NNS	noun, plural	<i>llamas</i>	VBD	verb past tense	<i>ate</i>	,	comma	<i>,</i>
NNP	proper noun, sing.	<i>IBM</i>	VBG	verb gerund	<i>eating</i>	.	sent-end punc	<i>. ! ?</i>
NNPS	proper noun, plu.	<i>Carolinas</i>	VBN	verb past part.	<i>eaten</i>	:	sent-mid punc	<i>: ; ... - -</i>

PoS Tagging: Python Example

- The code for PoS tagging using the TextBlob library:
 - `text = 'Google is looking at buying U.K. startup for $1 billion'`
 - `TextBlob(text).tags`
 - Output
 - `[('Google', 'NNP'),`
 - `('is', 'VBZ'),`
 - `('looking', 'VBG'),`
 - `('at', 'IN'),`
 - `('buying', 'VBG'),`
 - `('U.K.', 'NNP'),`
 - `('startup', 'NN'),`
 - `('for', 'IN'),`
 - `('1', 'CD'),`
 - `('billion', 'CD')]`

PoS Tagging: Methods and Evaluation

- **Methods:**
 - Hidden Markov Models (HMM)
 - Maximum Entropy Markov Models
 - Conditional Random Fields (CRF)
 - RNNs, Transformers → Recent Approaches
- **Evaluation:**
 - Accuracy
 - Macro-F1 (giving equal importance to each tag)

Text Pre-processing/Normalization:

Named Entity Recognition (NER)

- *Named entity recognition* (NER) is an optional next step in data preprocessing that seeks to locate and classify named entities in text into predefined categories.
- These categories can include names of persons, organizations, locations, expressions of times, quantities, monetary values, or percentages.
- While parts of speech are generally assigned to individual words or morphemes, a proper name is often an entire multiword phrase, like the name “Marie Curie”, the location “New York City”, or the organization “Stanford University”.
- We’ll use the term named entity for, roughly speaking, anything that can be referred to with a proper name: a person, a location, an organization, although as we’ll see the term is commonly extended to include things that aren’t entities per se.

NER: Python Example

- The NER performed using spaCy library:
 - text = 'Google is looking at buying U.K. startup for \$1 billion'
 - **for** entity **in** nlp(text).ents:
 - print("Entity: ", entity.text)
 - Output
 - Entity: Google
 - Entity: U.K.
 - Entity: \$1 billion
- Visualizing named entities in text using the displacy module, can also be incredibly helpful in speeding up development and debugging the code and training process:
 - from spacy import displacy
 - displacy.render(nlp(text), style="ent", jupyter = True)

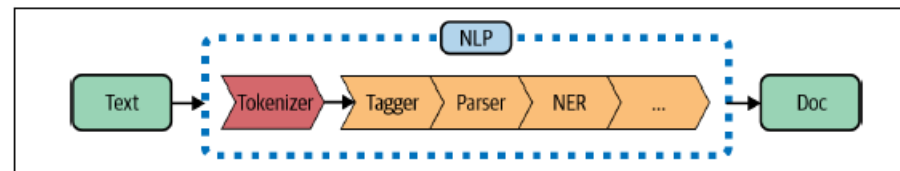
Google **ORG** is looking at buying **U.K. GPE** startup for **\$1 billion MONEY**

Text Pre-processing: Other optional preprocessing steps

- *Dependency parsing*
 - Assigns a syntactic structure to sentences to make sense of how the words in the sentence relate to each other.
- *Coreference resolution*
 - The process of connecting tokens that represent the same entity.
 - It is common in languages to introduce a subject with their name in one sentence and then refer to them as him/her/it in subsequent sentences.
- *Triplet extraction*
 - The process of recording subject, verb, and object triplets when available in the sentence structure.
- *Relation extraction*
 - A broader form of triplet extraction in which entities can have multiple interactions.

All Preprocessing in one go: Python Example

- When we call *nlp* on a text, spaCy first tokenizes the text to produce a *Doc* object.
- The *Doc* is then processed in several different steps.
- This is also referred to as the *processing pipeline*. The pipeline used by the default models consists of a *tagger*, a *parser*, and an *entity recognizer*.
- Each pipeline component returns the processed *Doc*, which is then passed on to the next component



All Preprocessing in one go: Python Example

- `text = 'Google is looking at buying U.K. startup for $1 billion'`
- `doc = nlp(text)`
- `pd.DataFrame([[t.text, t.is_stop, t.lemma_, t.pos_] for t in doc], columns=['Token', 'is_stop_word', 'lemma', 'POS'])`

– Output:

	Token	is_stop_word	lemma	POS
0	Google	False	Google	PROPN
1	is	True	be	VERB
2	looking	False	look	VERB
3	at	True	at	ADP
4	buying	False	buy	VERB
5	U.K.	False	U.K.	PROPN
6	startup	False	startup	NOUN
7	for	True	for	ADP
8	\$	False	\$	SYM
9	1	False	1	NUM
10	billion	False	billion	NUM

1 (Asst. Prof., ICT, PDEU,
Mandhinagar)

Example: Tokenization with BPE

Training Corpus

Word	Frequency
read	3
reads	2
reader	2
reading	2
unread	1
readers	1

(1) Represent each word as characters followed by _

(2) BPE Training: Perform exactly **6 merges**

- At each training step, count all adjacent token pairs weighted by word frequency.
- Merge the most frequent pair.
- If there is a tie, choose the **leftmost** pair from the word read if possible; otherwise choose alphabetically.

(3) Fill the table after each merge.

Merge #	Most frequent pair	New token
1	_____	_____
2	_____	_____
3	_____	_____
4	_____	_____
5	_____	_____
6	_____	_____

Example: Tokenization with BPE

(1) _ , a, d, e, g, i, n, r, s, u

(2)

Merge #	Most frequent pair	New token
1	(r, e)	re
2	(re, a)	rea
3	(rea, d)	read
4	(read, _)	read_
5	(read, e)	reade
6	(reade, r)	reader

Example: Tokenization with BPE

Testing on New Words

Test word	In training set?
read	Yes
reader	Yes
readers	Yes
reading	Yes
unread	Yes
readable	No
reread	No
unreader	No

(1) For each word, write the initial character form and the final BPE tokenization.

During testing, apply the learned merges in the same order.

(2) Choose any **three** tokenized test words, including at least **one OOV word**, and decode them.

During decoding, concatenate tokens and remove _.

Test word	Initial character form	Final BPE tokens
read	read _	read_
reader	reader _	reader _
readers	readers _	readers _
reading	reading _	reading _
unread	unread _	unread_
readable	readable _	readable _
reread	reread _	reread_
unreader	unreader _	unreader _

Chosen word	Token sequence	Decoded string	Matches original?
1.	read_	read	Yes
2.	unreader _	unreader	Yes
3.	readable _	readable	Yes

EU,

- In the test word – readable; the character b and l are not in vocabulary, still it is able to tokenize???
- **Yes — that’s exactly one of the key properties of BPE.**
- **Tokenizing OOV words falls back to characters.**
For readable:
 - Start from chars: r e a d a b l e _
 - Apply merges where possible: r e → re, re a → rea, rea d → read
⇒ read a b l e _
 - No merge exists for a b, b l, l e, so BPE leaves them as character tokens:
⇒ final tokens: read a b l e _
 - Here b and l are used as **base (character) tokens**. Even if they never appeared in a learned subword, they remain in the vocabulary.

THE END